

Hessian-Inverse-Free Prediction-Correction Method for Time-Varying Convex Optimization

Zhonghao Lin

(Supervised by **Dr. Jie Hou** and **Prof. Xianlin Zeng**)

Beijing Institute of Technology (BIT)

Presentation for IEEE CDC 2026





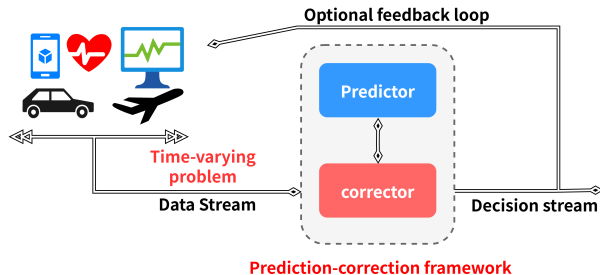
- ① Background and Problem
- ② Method Design
- ③ Main Theoretical Results
- ④ Numerical Results

Time-Varying Optimization: Introduction

Time-varying optimization

The objective evolves continuously, and the algorithm should track the optimizer trajectory in real time.^a

^aSimonetto A, et al. Time-Varying Convex Optimization: Time-Structured Algorithms and Applications[J]. *Proceedings of the IEEE*, 2020, 108(11): 2032-2048.



- $x^*(t)$ is a **moving target**.
- Decisions allow only **limited computation**.
- Temporal information can **predict** optimizer motion.

Goal: online tracking, not one-shot optimization.



Continuous-time time-varying optimization

$$x^*(t) = \arg \min_{x \in \mathbb{R}^n} f(x, t), \quad t \geq 0.$$

Strong convexity with possible ill-conditioning

$$0 < mI_n \preceq \nabla_{xx} f(x, t) \preceq LI_n,$$

where $m > 0$ may be very small.

Flow-based algorithm

$$\dot{x}(t) = \Phi(x(t), t), \quad x(0) = x_0.$$

Structure of prediction–correction flow

$$\dot{x}(t) = \underbrace{\Phi_{\text{pred}}(x(t), t)}_{\text{prediction}} + \underbrace{\Phi_{\text{corr}}(x(t), t)}_{\text{correction}}.$$

- 1 **Prediction:** captures the trend of $x^*(t)$.
- 2 **Correction:** applies a standard optimization flow to the current objective, such as **gradient flow** or **Newton flow**.

Correction-Only Flow: A Toy Example

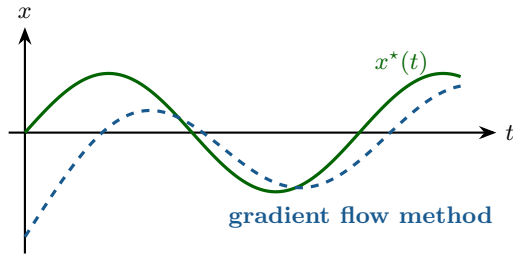
A Scalar Time-Varying Problem

Consider

$$f(x, t) = \frac{1}{2}(x - x^*(t))^2, \quad x^*(t) = \sin t.$$

The correction-only **gradient flow** is

$$\dot{x} = -\beta(x - x^*(t)), \quad \beta > 0.$$



Observed Phenomenon

The transient error decays, but the trajectory still keeps a small asymptotic tracking error from the moving optimizer.

Why does correction-only gradient flow fail to achieve unbiased asymptotic tracking?



Why Prediction Is Needed

A Scalar Time-Varying Problem

Consider

$$f(x, t) = \frac{1}{2}(x - x^*(t))^2, \quad x^*(t) = \sin t.$$

The correction-only **gradient flow** is

$$\dot{x} = -\beta(x - x^*(t)), \quad \beta > 0.$$

Define the tracking error and differentiate it:

$$e(t) = x(t) - x^*(t),$$

$$\dot{e}(t) = -\beta e(t) - \dot{x}^*(t) = -\beta e(t) - \cos t.$$

For this example, the asymptotic tracking error is

$$\limsup_{t \rightarrow \infty} |x(t) - x^*(t)| = \frac{1}{\sqrt{\beta^2 + 1}} = \mathcal{O}(1/\beta).$$

- ① Larger β reduces the error bound.
- ② Finite β does not eliminate the error.
- ③ A prediction term is needed to compensate temporal variation.

Then, how to design a prediction term to achieve unbiased tracking?



Classical Newton Prediction-Correction Flow

Differentiating the optimality condition along $x^*(t)$ gives

$$\nabla_{xx}f(x^*(t), t)\dot{x}^*(t) + \nabla_{xt}f(x^*(t), t) = 0 \implies \dot{x}^*(t) = -\nabla_{xx}^{-1}f(x^*(t), t)\nabla_{xt}f(x^*(t), t).$$

This motivates **Newton-PC**, which combines a **prediction** term and a **correction** term.¹

$$\dot{x}(t) = \underbrace{-\nabla_{xx}^{-1}f(x(t), t)\nabla_{xt}f(x(t), t)}_{\text{prediction}} - \underbrace{\beta\nabla_{xx}^{-1}f(x(t), t)\nabla_x f(x(t), t)}_{\text{correction}}.$$

- ① **Strength:** Under suitable regularity conditions, Newton-PC achieves **unbiased asymptotic tracking**. Thus, $\limsup_{t \rightarrow \infty} \|x(t) - x^*(t)\| = 0$.
- ② **Bottleneck:** Online inverse-Hessian computation is costly and numerically sensitive.

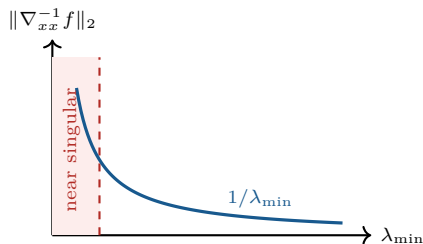
¹Fazlyab M, et al. Prediction-Correction Interior-Point Method for Time-Varying Convex Optimization[J]. IEEE Transactions on Automatic Control, 2018, 63(7): 1973-1986.

The Inverse-Hessian Bottleneck Near Singularity

Observe that $\|\nabla_{xx}^{-1} f(x, t)\|_2 = \frac{1}{\lambda_{\min}(\nabla_{xx} f(x, t))} \leq \frac{1}{m}$.

When $m \ll 1$, the inverse Hessian can have an extremely large spectral norm.

- Large inverse gains amplify numerical errors.
- Repeated inversion or factorization is unattractive in real-time settings.
- Classical Newton-PC becomes sensitive near singularity.



Key Message

A small minimum Hessian eigenvalue leads to a large inverse-Hessian gain.

Main Idea: Replace the Inverse Hessian by $H(t)$

Introduce an auxiliary matrix state

$$H(t) \approx \nabla_{xx}^{-1} f(x(t), t).$$

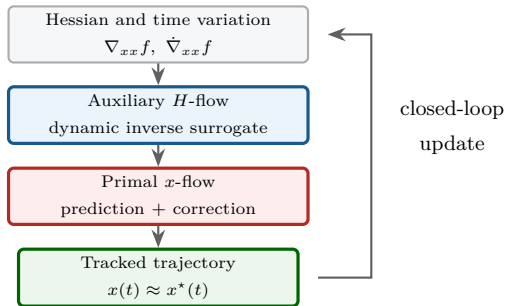
Then use the prediction–correction flow

$$\dot{x}(t) = -H(t) [\beta \nabla_x f(x(t), t) + \nabla_{xt} f(x(t), t)].$$

Here $\beta > 0$ is the primal correction gain.

Desired behavior of $H(t)$

- approximate inverse-Hessian information;
- remain **positive definite** and **bounded**;
- avoid excessive gain near singularity.



$H(t)$ replaces $\nabla_{xx}^{-1} f$ without explicitly computing it.

Constructing the H -Flow I: Prediction Term

If $H(t) = \nabla_{xx}^{-1} f(x(t), t)$, then

$$\nabla_{xx} f(x(t), t) H(t) = I_n.$$

Differentiating this inverse relation suggests

$$\dot{H}_{\text{pred}} = -H \dot{\nabla}_{xx} f(x(t), t) H.$$

The term predicts how inverse-Hessian information changes with time.

It is the **prediction term** in the H flow.

Example: ZNN-style inverse tracking

For a time-varying matrix equation

$$M(t)Y(t) = I_n,$$

one obtains

$$\dot{Y}(t) = -Y(t)\dot{M}(t)Y(t).$$

Dynamic tracking replaces repeated online inversion.^a

^aZhang Y, et al. Design and Analysis of a General Recurrent Neural Network Model for Time-Varying Matrix Inversion[J]. IEEE Transactions on Neural Networks, 2005, 16(6):1477-1490.

How Is the Hessian Derivative Obtained?

The term $\dot{\nabla}_{xx}f(x(t), t)$ is the **total derivative along the solution trajectory** $(x(t), H(t))$..

Analytical expression

$$\frac{d}{dt}\nabla_{xx}f(x(t), t) = \sum_{i=1}^n \nabla_{xxx_i}f(x(t), t)\dot{x}_i(t) + \nabla_{xxt}f(x(t), t).$$

Note: The analysis assumes this total derivative is bounded along the solution trajectory.

Numerical Implementation

For numerical simulation, the continuous-time flow can be discretized by forward Euler with sampling period h . If the Hessian derivative is not available in closed form, it can be approximated by a first-order backward difference:

$$\dot{\nabla}_{xx}f(x_k, t_k) \approx \frac{\nabla_{xx}f(x_k, t_k) - \nabla_{xx}f(x_{k-1}, t_{k-1})}{h}.$$



Constructing the H -Flow II: Correction Term

Consider the “inverse residual”

$$I_n - \nabla_{xx} f(x(t), t) H(t).$$

A multiplication-based correction is

$$\dot{H}_{\text{corr}} = \alpha H [I_n - \nabla_{xx} f(x(t), t) H].$$

Here $\alpha > 0$ is the inverse-correction gain. It reduces the mismatch between $H(t)$ and the inverse Hessian.

It is the **correction term** in the H flow.

Example: Newton–Schulz analogy

For a fixed matrix, a classical inverse iteration is

$$H^+ = H(2I_n - \nabla_{xx} f H).$$

The update ($\Delta H := H^+ - H$) contains

$$\Delta H = H(I_n - \nabla_{xx} f H).$$

This has the same residual structure.^a

^aHigham N J. Functions of Matrices: Theory and Computation [M]. SIAM, 2008.

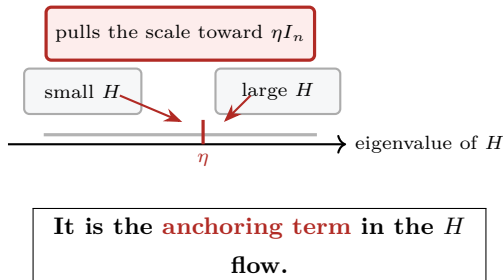
Constructing the H -Flow III: Anchoring Term

When the Hessian is nearly singular, an unbiased inverse surrogate may have a very large norm.

To regulate the scale of $H(t)$, add an **anchoring term**

$$\dot{H}_{\text{anc}} = \gamma H(\eta I_n - H).$$

- $\gamma \geq 0$: anchoring strength;
- $\eta > 0$: anchoring scale.

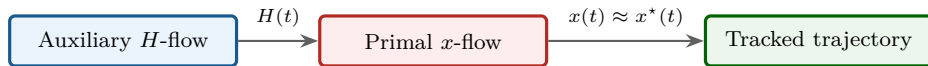


Combining prediction, correction, and anchoring terms, we obtain:

$$\dot{H} = \dot{H}_{\text{pred}} + \dot{H}_{\text{corr}} + \dot{H}_{\text{anc}}.$$

Auxiliary H -flow: $\dot{H} = \underbrace{-H\dot{\nabla}_{xx}f(x(t),t)H}_{\text{prediction}} + \underbrace{\alpha H[I_n - \nabla_{xx}f(x(t),t)H]}_{\text{correction}} + \underbrace{\gamma H(\eta I_n - H)}_{\text{anchoring}},$

Primal x -flow: $\dot{x} = \underbrace{-H\nabla_{xt}f(x(t),t)}_{\text{prediction}} + \underbrace{-\beta H\nabla_x f(x(t),t)}_{\text{correction}}.$



The coupled flow avoids explicitly computing $\nabla_{xx}^{-1}f(x(t),t)$.



Property I: Positive Definiteness of $H(t)$

H-flow property 1: the auxiliary matrix state remains **positive definite**.

Assumption 1

There exist constants $0 < m \leq L$ such that

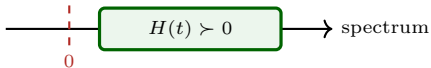
$$mI_n \preceq \nabla_{xx} f(x, t) \preceq LI_n.$$

Lemma 1: positive definiteness

Under **Assumption 1** and $H(0) \in \mathbb{S}_{++}^n$,

$$H(t) \in \mathbb{S}_{++}^n, \quad \forall t \geq 0.$$

Initialization: $H(0) \in \mathbb{S}_{++}^n$.



Why property 1 useful? The intended inverse Hessian surrogate should inherit the same positive-definite structure as an inverse Hessian in strongly convex optimization.

Property II: Uniform Boundedness of $H(t)$

H-flow property 2: the spectrum of $H(t)$ admits a **uniform upper bound**.

Assumptions used

Assumption 1:

$$mI_n \preceq \nabla_{xx} f(x, t) \preceq LI_n.$$

Assumption 2: along the solution trajectory,

$$\|\nabla_{xt} f(x(t), t)\|_2 \leq M_b, \quad \|\dot{\nabla}_{xx} f(x(t), t)\|_2 \leq M_A.$$

Key condition

$$\alpha m + \gamma > M_A.$$

Lemma 2: spectral boundedness

Under **Assumptions 1–2** and the **key condition**,

$$\|H(t)\|_2 \leq H_1, \quad \forall t \geq 0,$$

$$\text{where } H_1 = \max \left\{ \lambda_{\max}(H(0)), \frac{\alpha + \gamma \eta}{\alpha m + \gamma - M_A} \right\}.$$

Why property 2 useful? The bound H_1 gives an explicit way to regulate the spectrum of $H(t)$, compensating for the large inverse-Hessian gain when m is small.



From Inverse-Hessian Bias to Tracking Error

Define $\rho := \max\{|1 - \eta m|, |1 - \eta L|\}$.

Then the “asymptotic inverse-tracking bias” satisfies

$$\limsup_{t \rightarrow \infty} \|H^{-1}(t) - \nabla_{xx} f(x(t), t)\| \leq \frac{\gamma \rho}{\alpha + \gamma \eta}.$$

- $\gamma = 0$: no asymptotic inverse-tracking bias.
- $\gamma > 0$: bounded but generally nonzero bias.

Theorem: optimization tracking bound

Let

$$R_{\infty} = \frac{\gamma \rho \sqrt{n}}{\alpha + \gamma \eta} H_1.$$

If $R_{\infty} < 1$, then

$$\limsup_{t \rightarrow \infty} \|x(t) - x^*(t)\|_2 \leq \frac{R_{\infty}}{m\beta(1 - R_{\infty})} M_b.$$

The inverse-tracking bias is explicitly propagated to the optimizer-tracking error.

Anchoring Trade-off

Without anchoring: $\gamma = 0$

If

$$\alpha m > M_A,$$

then

$$R_\infty = 0, \quad x(t) \rightarrow x^*(t).$$

Exact asymptotic tracking.

With anchoring: $\gamma > 0$

The boundedness condition becomes

$$\alpha m + \gamma > M_A.$$

Then $H(t)$ is better regulated, but generally

$$R_\infty > 0.$$

Practical tracking with spectral robustness.

Anchoring trade-off: Without anchoring, $\alpha m > M_A$ yields $R_\infty = 0$ and exact asymptotic tracking. With $\gamma > 0$, the condition relaxes to $\alpha m + \gamma > M_A$ and $H(t)$ is better regulated, while R_∞ is generally nonzero and only practical tracking is guaranteed.



Numerical Example: Nearly Singular Strongly Convex Quadratic

Problem

$$f(x, t) = \frac{1}{2}(x - c(t))^{\top} \nabla_{xx} f(t)(x - c(t)), \quad x^*(t) = c(t).$$

The Hessian is designed so that

$$\lambda_{\min}(\nabla_{xx} f(t)) \approx 10^{-6}.$$

Thus, the problem is **strongly convex** but **nearly singular**.

Simulation setting:

$$x, c \in \mathbb{R}^6, \quad \text{stepsize } h = 10^{-4}.$$

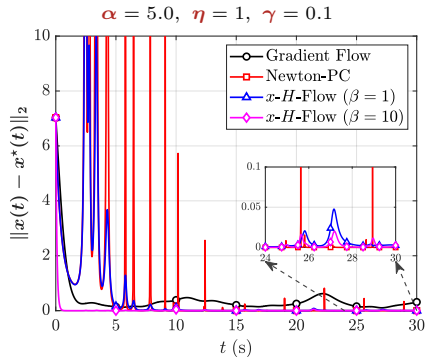
Compared flows

- ① **Gradient flow:** $\dot{x} = -\beta \nabla_x f.$
- ② **Newton-PC flow:** $\dot{x} = -\nabla_{xx}^{-1} f(\beta \nabla_x f + \nabla_{xt} f).$
- ③ **Proposed x - H flow:**

$$\begin{cases} \dot{x} = -H(\beta \nabla_x f + \nabla_{xt} f), \\ \dot{H} = -H \dot{\nabla}_{xx} f H + \alpha H (I_n - \nabla_{xx} f H) \\ \quad + \gamma H (\eta I_n - H). \end{cases}$$

Numerical Results: Tracking and Spectral Regulation

Simulation results – tracking error



(i) Larger β gives a smaller asymptotic tracking error.

Simulation results – spectral norm

$$\alpha = 5.0, \eta = 1, \beta = 1$$

Method	Quantity	Max. norm
Newton-PC	$\ \nabla_{xx}^{-1} f\ _2$	9.99×10^5
x -H, $\gamma = 0.05$	$\ H(t)\ _2$	1.15×10^2
x -H, $\gamma = 0.10$	$\ H(t)\ _2$	5.81×10^1
x -H, $\gamma = 0.20$	$\ H(t)\ _2$	2.95×10^1

(ii) Larger γ reduces the max spectral norm.

(iii) The proposed flow is smoother near ill-conditioning than Newton-PC.

Thanks for listening!

